

CSCI 135 Assignment – Command Line Utilities

CSCI 135 Assignment – Command Line Utilities

September 6th / September 8th, 2023

Due Date: Sunday, 9/10/2023 11:59PM

Points: 20

Topic: Command Line Utilities for Type Conversion (aka Time to Swim)

The goal of this assignment is to apply what you have learned in prior labs and some new material. You will create several Python scripts that can be used as command line utilities to convert decimal numbers to/from binary (base 2) and hexadecimal (base 16).

Submission:

After following the instructions below, please submit the four specified Python scripts in Moodle. Upload these as separate files – do not zip them. Be sure each script has the recommended header including the file name, your name, and a description of the program. Also include an “Example Usage” section to show an example of how to call each individual script from the command line.

- DecimalToBinary.py
- BinaryToDecimal.py
- DecimalToHex.py
- HexToDecimal.py

Grading:

Each program/script is worth 5 points, for a total of 20 points. For each script, you will be graded according to the following criteria:

	<i>ScriptName.py</i>
Header Comment	1
Program Written Correctly, Following Direction	2
Program Runs Correctly, Including Correct Output	2
Total	5

Introduction

Command Line Utilities, programs that can be run from the command line, are a powerful component of the Linux operating system and Unix prior to that. Microsoft Windows (and MS-DOS prior) borrowed this idea to produce their own utilities, and Mac-OS follows similarly since it is also based on a derivative of Unix. In this lab assignment, you will create several Python scripts that serve as command line utilities to convert decimal numbers to/from binary (base 2) and hexadecimal (base 16).

Have you ever heard the saying, “Sink or swim!” This saying comes from the old way of teaching someone to swim by throwing them into the pool/river/lake to “teach” them how to swim out of necessity. Of course, we are not so reckless in teaching programming in this class, but at some point, you need to be able to jump in and write programs without being told step-by-step what to do. This requires that you use what you have learned previously, find and learn some new things, and recognize what is useful and what is not in solving the task at hand.

In this lab assignment I will show and explain how to write the first script. You will then have to find a solution for the second script on your own using material from class and our book(s). Then you will write the third and fourth scripts using the first two as examples.

You can do it! It might be scary at first, particularly if you have no idea what to do. But once you figure it out, you will gain confidence and will be ready for subsequent challenges.

Part 1: DecimalToBinary

Create a script named `DecimalToBinary.py` that takes one command line argument, a whole number (positive integer), and outputs the corresponding binary (base 2) representation. I.e., it converts a decimal number to binary. (NOTE: I will show you exactly how to do this. Follow the subsequent steps. In fact, I will give you the code.)

Here is an example run of the script:

```
C:\Users\dgalarus> python DecimalToBinary.py 25
```

Output:

```
11001
```

Here are several more example runs and output:

```
C:\Users\dgalarus> python DecimalToBinary.py 2020
```

```
11111100100
```

```
C:\Users\dgalarus> python DecimalToBinary.py 100
```

```
1100100
```

CSCI 135 Assignment – Command Line Utilities

Here is the code for DecimalToBinary.py. Use it! With that said, you should acknowledge in the header that I am the original author and that you are using code I provided to you. You should include your name in the header too.

```
#####  
#  
# Filename: DecimalToBinary.py  
#  
# Author: Doug Galarus  
#  
# Description: Takes a whole number (positive integer),  
# provided as a command line argument, and outputs the  
# binary representation of that same number.  
#  
# Example use:  
#  
#     python DecimalToBinary.py 25  
#  
#####  
  
import sys  
  
print(bin(int(sys.argv[1]))[2:])
```

Try it out! Use it on the examples shown above and see that it gives the expected output. Try some other inputs. Make sure it works as specified!

How does it work?

Aside from the header, the `import` statement, and a few blank line between, this is a one-liner. Quite a bit is packed into this one-liner. I don't want to set a bad example here. Be careful with trying to pack too much into one line. For our purposes, this one is ok. Let's unpack it from the inside out:

We first get the command line argument, similar to what we did in the last assignment:

```
print(bin(int(sys.argv[1]))[2:])
```

Next, we convert it to an integer. Remember that command line arguments are passed to our scripts as strings/text:

```
print(bin(int(sys.argv[1]))[2:])
```

CSCI 135 Assignment – Command Line Utilities

Now, we convert the integer to its binary representation using the `bin` function. The result is a string (text) containing the binary representation, as well as `'0b'` at the start.

```
print(bin(int(sys.argv[1]))[2:])
```

We do not want to output the `'0b'` prefix. We just want to output the 0s and 1s that are the binary representation. The following indexing of the string gives the text starting with the character at index 2. In other words, it gives the substring minus the first two characters:

```
print(bin(int(sys.argv[1]))[2:])
```

Finally, we print the result:

```
print(bin(int(sys.argv[1]))[2:])
```

That was a lot, and several new things were used here: nested function calls and substrings. The good news is you have been given the code.

Try it and understand it before moving on.

Upload DecimalToBinary.py to Moodle as the first part of your submission for this assignment.

Part 2: BinaryToDecimal

Create a script named `BinaryToDecimal.py` that takes one command line argument, the binary (base 2) representation of a whole number (positive integer), and outputs the corresponding decimal representation. I.e., it converts a binary number to decimal.

Here is an example run of the script:

```
C:\Users\dgalarus> python BinaryToDecimal.py 11001
```

Output:

```
25
```

Here are several more example runs and output:

```
C:\Users\dgalarus> python BinaryToDecimal.py 11111100100
```

```
2020
```

```
C:\Users\dgalarus> python BinaryToDecimal.py 1100100
```

```
100
```

Good news! The (expected) code for this part is also a one-liner and is simpler than the code I gave you for Part 1. Bad news! I do not give you this code.

Good news! The function you need to use to make this is shown in Chapter 3 of our (optional) Lubanovic book and in my slides for Friday showing that same content. (I will have posted those slides by the time you read this. Look at them ahead of time.) Find and use that function along with a print statement and retrieval of the command line argument. You are not expected to write or implement an algorithm for these conversions beyond what is described here.

Test your script thoroughly to ensure that it works. Make sure you include a proper header.

Upload BinaryToDecimal.py to Moodle as the second part of your submission for this assignment.

Part 3: DecimalToHex

Create a script named `DecimalToHex.py` that takes one command line argument, a whole number (positive integer), and outputs the corresponding hexadecimal (base 16) representation. I.e., it converts a decimal number to hex.

Here is an example run of the script:

```
C:\Users\dgalarus> python DecimalToHex.py 25
```

Output:

```
19
```

Here are several more example runs and output:

```
C:\Users\dgalarus> python DecimalToHex.py 2020
```

```
7e4
```

```
C:\Users\dgalarus> python DecimalToHex.py 100
```

```
64
```

Note: When used for hexadecimal (base 16) numbers, the letters A-F can be expressed using either upper or lower case. For this exercise, lower case will be used because it is used by Python by default, and we have not covered conversion to upper case yet.

Hint: If you did Part 1 (`DecimalToBinary.py`) right, this script can be done as a simple modification of that script. Be careful. Make sure you don't modify that original script. Instead, make a copy, name it correctly for this part, and make the modification.

Test your script thoroughly to ensure that it works. Make sure you include a proper header, specific to this new script.

Upload `DecimalToHex.py` to Moodle as the third part of your submission for this assignment.

Part 4: HexToDecimal

Create a script named `HexToDecimal.py` that takes one command line argument, the hexadecimal (base 16) representation of a whole number (positive integer), and outputs the corresponding decimal representation. I.e., it converts a hexadecimal number to decimal.

Here is an example run of the script:

```
C:\Users\dgalarus> python HexToDecimal.py 19
```

Output:

```
25
```

Here are several more example runs and output:

```
C:\Users\dgalarus> python HexToDecimal.py 7E4
```

```
2020
```

```
C:\Users\dgalarus> python HexToDecimal.py 64
```

```
100
```

Hint: If you did Part 2 (`BinaryToDecimal.py`) right, this script can be done as a simple modification of that script. Be careful. Make sure you don't modify that original script. Instead, make a copy, name it correctly for this part, and make the modification.

Test your script thoroughly to ensure that it works. Make sure you include a proper header, specific to this new script.

Upload HexToDecimal.py to Moodle as the fourth part of your submission for this assignment.

EXTENSIONS

You will not submit the answers to these extensions as part of your assignment submission. You should work through and understand them to learn more and practice. You may be held responsible for some of the content within, so you really should look through them ...

NOTE: Be sure to follow the guidance given above for this lab and the associated scripts that you will submit. The following exercises give a different perspective on number systems and conversions between number systems.

From the Liang book - NOTE: We will **not** use MyProgrammingLab in the book. You should be able to do exercises from Liang without MyProgrammingLab.:

Some of these are hard and use material we have not covered yet and might not cover this semester. Take a look at them. Try some, but don't get frustrated. Try not to get confused by seeing different ways to solve a problem.

1. *Liang gives an in-depth treatment of Number Systems in Appendix C of his book. Read through Appendix C to better understand Binary and Hexadecimal numbers. Try some of the exercises to see if you can do the calculations by hand. Try the MS-Windows Calculator in Programmer mode to see how binary, hex and octal conversions and calculations are done.*
2. *Table 3.4 shows format specifiers for the format() function that will format numbers using binary, hex, and more. Investigate how this is done. This approach also yields some very short code to perform conversions between bases.*
3. *Look at GuessBirthday.py in Listing 4.3. Try it out. How and why does the binary representation of a number help in making this program work?*
4. *Exercises 8.8, 8.9 and 8.10 ask the reader to write functions that do these conversions from scratch. I.e., they require the user to write code to do these conversions without using built-in shortcuts like we used above. How would do this?*
5. *The word Binary gets used in many places in the Liang book including in reference to topics that go beyond representations of numbers with different bases. What are some of these uses, and why is Binary part of their names?*
6. *Exercises 15.19, 15.20, 15.21, 15.22 revisit the problem of doing conversions, this time using recursion. What is recursion and how does it help to solve this problem?*

CSCI 135 Assignment – Command Line Utilities

From the *Things to Do* section at the end of Chapter Two of our optional Lubanovic book:

- 3.1 How many seconds are in an hour? Use the interactive interpreter as a calculator and multiply the number of seconds in a minute (60) by the number of minutes in an hour (also 60).
- 3.2 Assign the result from the previous task (seconds in an hour) to a variable called `seconds_per_hour`.
- 3.3 How many seconds are in a day? Use your `seconds_per_hour` variable.
- 3.4 Calculate seconds per day again, but this time save the result in a variable called `seconds_per_day`.
- 3.5 Divide `seconds_per_day` by `seconds_per_hour`. Use floating-point (/) division.
- 3.6 Divide `seconds_per_day` by `seconds_per_hour`, using integer (//) division. Did this number agree with the floating-point value from the previous question, aside from the final .0?

From me:

1. Why are command line arguments passed to programs as strings/text even when they are numbers?
2. A desirable feature of command line utilities is that they can be “chained” together by sending (piping) the output from one as the input into another. For instance, we could create a HexToBinary converter by chaining together `HexToDecimal.py` and `DecimalToBinary.py`. Unfortunately, this won’t work with the scripts from this lab as-is. Why not? What could be done to fix this?
3. Experiment with the scripts you wrote for this assignment. Use different numbers. Can you convert between binary and decimal by hand or in your head? How about hex and decimal?